Heap Data Structures: Implementation, Analysis, and Applications

Rachana Peddi

University of the Cumberlands

MSCS-532: Algorithms and Data Structures

Assignment 4

Fall 2026

Heap Data Structures: Implementation, Analysis, and Applications

Introduction

Heap data structures provide efficient support for sorting and priority-based processing. This project implements Heapsort and a max-priority queue in Python, analyzes their asymptotic behavior, compares Heapsort empirically with Quicksort and Merge Sort, and demonstrates a task-scheduling application. The implementation uses Python lists to represent binary heaps because array-based storage provides direct parent-child indexing without the pointer overhead required by an explicit tree representation. The completed tests also include edge cases so that correctness is evaluated in addition to asymptotic efficiency.

Heapsort Implementation

The Heapsort implementation follows the standard two-stage design. First, the input list is transformed into a max-heap. In a zero-based array, the children of an element at index i are located at $2i + 1$ and $2i + 2$. After the max-heap is built, the root contains the largest element. The algorithm repeatedly exchanges the root with the last element in the active heap, reduces the heap size, and restores the max-heap property with a downward heapify operation. This process places the largest remaining value into its final sorted position during each extraction (Cormen et al., 2022).

The implementation was tested with random integers, already sorted input, reverse-sorted input, duplicate values, negative values, mixed positive and negative values, a single element, an empty list, and all-equal values. All nine displayed tests returned True. These cases address the rubric requirement that the implementation handle edge conditions rather than only typical input.

Heapsort Analysis

Building a max-heap from an arbitrary array requires O(n) time. Although a simple multiplication of n nodes by O(log n) heapify time would suggest O(n log n), that bound is not tight because most nodes occur near the leaves and require little or no downward movement. The aggregate cost of heap construction is linear (Cormen et al., 2022).

After construction, Heapsort performs n - 1 extraction steps. Each extraction swaps the root with the final element in the active heap and can require a heapify operation through a heap of height O(log n). Therefore, the sorting phase requires O(n log n) time. Combining O(n) heap construction with O(n log n) extraction yields O(n log n) overall.

Case	Time Complexity	Reason
Best case	O(n log n)	The extraction phase still performs repeated heap restoration.
Average case	O(n log n)	Heap height remains logarithmic and n extractions are performed.
Worst case	O(n log n)	Each extraction can require movement through the heap height.

Unlike algorithms whose running time can change substantially with the original ordering of the input, Heapsort retains the same asymptotic O(n log n) bound for best, average, and worst cases. The array-based algorithm can sort in place, so its auxiliary storage is O(1), excluding implementation-level call-stack overhead if heapify is written recursively. The Python implementation used for this project therefore has predictable asymptotic behavior and does not require an additional array proportional to n for the sorting process.

Empirical Comparison of Sorting Algorithms

The performance experiment compared Heapsort, Quicksort, and Merge Sort over five trials for each measurement. Input sizes of 100, 500, 1,000, 2,000, 5,000, and 10,000 were tested

under random, sorted, and reverse-sorted distributions. Before timing, the program also performed correctness and edge-case verification all three algorithms passed those checks. The measurements below are empirical results from the executed project and can vary across computers and runs.

Distribution	n	Heapsort (s)	Quicksort (s)	Merge Sort (s)
Random	100	0.000115	0.000078	0.000096
Random	500	0.000610	0.000422	0.000612
Random	1,000	0.001707	0.001102	0.001456
Random	2,000	0.003498	0.002331	0.002885
Random	5,000	0.010595	0.006626	0.008384
Random	10,000	0.032304	0.014149	0.017253

Distribution	n	Heapsort (s)	Quicksort (s)	Merge Sort (s)
Sorted	100	0.000111	0.000055	0.000078
Sorted	500	0.000752	0.000321	0.000454
Sorted	1,000	0.001696	0.000615	0.000904
Sorted	2,000	0.003625	0.001539	0.002220
Sorted	5,000	0.010559	0.004534	0.005571
Sorted	10,000	0.028669	0.009863	0.011698

Distribution	n	Heapsort (s)	Quicksort (s)	Merge Sort (s)
Reverse sorted	100	0.000094	0.000056	0.000086
Reverse sorted	500	0.000645	0.000322	0.000466
Reverse sorted	1,000	0.001495	0.000639	0.001080
Reverse sorted	2,000	0.003161	0.001393	0.002085
Reverse sorted	5,000	0.009525	0.004090	0.005328
Reverse sorted	10,000	0.022135	0.009886	0.011473

The measured times increase with input size for all three algorithms, which is consistent with the expected growth of comparison-based O(n log n) sorting methods. In this specific experiment, Quicksort had the lowest measured time at n = 10,000 for all three distributions, followed by Merge Sort and then Heapsort. For random input at n = 10,000, the measured times were 0.014149 seconds for Quicksort, 0.017253 seconds for Merge Sort, and 0.032304 seconds for Heapsort. The same ordering appeared for sorted and reverse-sorted data in this run.

These measurements do not change the asymptotic analysis. Big-O notation describes growth as input size increases rather than guaranteeing that algorithms with the same asymptotic order will have identical wall-clock times. Constant factors, implementation details, memory access patterns, and the particular Quicksort pivot strategy can affect empirical performance. The results therefore support using theoretical complexity together with measured behavior rather than treating either one alone as a complete performance evaluation.

Priority Queue Design

The priority queue is implemented as a max-heap stored in a Python list. A max-heap was selected because the scheduling example processes the task with the numerically highest priority first. Each Task object stores a task identifier, a description, and a priority. The list representation is appropriate because it stores a complete binary tree compactly and supports constant-time index calculations for parent and child relationships.

The project demonstrates an empty queue, insertion of multiple tasks, retrieval of the highest-priority task, an increase-key operation, rejection of an invalid priority decrease, extraction in descending priority order, a single-task queue, and duplicate priorities. The observed extraction sequence [12, 10, 7, 4, 3] matched the expected sequence, and the duplicate-priority test also passed.

Priority Queue Operations and Complexity

Operation	Complexity	Explanation
is_empty()	O(1)	Checks whether the heap contains any elements.
peek_max()	O(1)	Reads the root of the max-heap without removing it.
insert(task)	O(log n)	Adds at the end and moves upward until the heap property is restored.

extract_max()	O(log n)	Removes the root, moves the last item to the root, and heapifies downward.
increase_key()	O(log n)*	After locating the task, the changed element can move upward through the heap height.

*The heap adjustment itself is O(log n). If an implementation searches a plain list linearly to locate a task by ID before changing its priority, that lookup contributes O(n) unless an auxiliary ID-to-index mapping is maintained. An index map can preserve efficient lookup while requiring updates whenever heap elements are swapped.

Effect of Heap Load

For a binary heap, increasing the number of stored tasks increases the heap height logarithmically because the structure is a complete binary tree. With n elements, the height is O(log n). Consequently, insertion, extraction, and heap-based priority adjustment remain logarithmic as the queue grows, while inspecting the root remains constant time. This differs from hash-table load factor a binary heap does not use a hash load-factor threshold. The practical effect of a larger queue is instead a gradual increase in heap height and memory usage proportional to the number of stored tasks.

Scheduler Simulation and Application

The scheduler simulation applies the max-priority queue to a task-processing scenario. Seven tasks were submitted with priorities 2, 6, 10, 3, 8, 1, and 7. The scheduler processed them in the verified order 10, 8, 7, 6, 3, 2, and 1, demonstrating that the highest-priority work is selected first regardless of submission order. The simulation also demonstrated a dynamic update a routine report task originally assigned priority 2 was increased to priority 10 after a production

issue made it urgent. The updated task then became the next task selected by the queue. Both the normal scheduling verification and dynamic-priority test returned True.

This model is applicable to systems in which work items have different urgency levels, such as application alerts, integration messages, high-priority data requests, backups, and routine reporting. A heap-based priority queue is useful because the scheduler can obtain the most urgent item efficiently without fully sorting the entire task collection after every insertion or priority change.

Code Quality and Documentation

The project separates concerns across heapsort.py, priority_queue.py, scheduler_simulation.py, and performance_test.py. The Heapsort module focuses on sorting and edge-case tests the priority-queue module encapsulates task and heap operations the scheduler simulation demonstrates an application of the queue and the performance module verifies correctness and collects empirical timing results. This organization supports readability, testing, and maintenance. The repository README should retain concise execution instructions for each script and summarize the major theoretical and empirical findings.

Conclusion

The project demonstrates both the theoretical and practical behavior of heap data structures. Heapsort provides O(n log n) running time in the best, average, and worst cases, with linear-time heap construction followed by logarithmic heap restoration during repeated extraction. The empirical experiment showed increasing execution time as input size grew and, for the tested implementation and machine, Quicksort produced the lowest measured times among the three algorithms at the largest tested size. The max-priority queue correctly supported

insertion, extraction, priority increases, empty-queue handling, and edge cases, while the scheduler demonstrated how those operations can support priority-driven processing.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.